ENGINEERING OPTIMIZATION
https://doi.org/10.1080/0305215X.2019.1691547

Taylor & Francis
Taylor & Francis Group

Check for updates

Multi-fidelity algorithms for the horizontal alignment problem in road design

Mahdi Aziz[a], Warren Hare [b], Majid Jaberipour[a] and Yves Lucet [a]

[a]Computer Science, Irving K. Barber School of Arts and Sciences/Computer Science, Mathematics, Physics, Statistics (IKBSAS/CMPS), University of British Columbia, Kelowna, Canada; [b]Mathematics, IKBSAS/CMPS, University of British Columbia, Kelowna, Canada

ABSTRACT
Multi-fidelity algorithms for solving the horizontal alignment problem in road design are considered. A multi-fidelity surrogate model is built and quantile regression is used to understand its accuracy at various fidelity levels. Two algorithms are compared: a generalized pattern search algorithm with adaptive precision control, and a trust-region algorithm for unconstrained problems with controlled error. To make a fair comparison, the parameters of each algorithm are tuned on five small roads using performance profiles. Then the algorithms are evaluated on 35 roads, ranging from small to very large roads. The results show that using multi-fidelity surrogates in optimization algorithms provide notable speed-up when compared to single-fidelity algorithms while preserving the quality of solutions (cost error $< 1\%$). On the longest roads, higher speed-up and better accuracy are observed.

ARTICLE HISTORY
Received 10 June 2019
Accepted 7 November 2019

KEYWORDS
Multi-fidelity; road design; surrogate; generalized pattern search; trust region

1. Introduction

In 2015/2016, Canada territorial, provincial and federal agencies spent 15.9 billion dollars to build and extend roads, and more than 50% of revenue gained by the territorial, provincial and federal government came from road transportation (Transportation 2017). Reducing such costs without compromising safety can produce savings in the millions for a single road. The five principal costs of roads are: planning, design and administrative costs; construction costs; maintenance costs; social, political and environmental costs; and user costs (Jha et al. 2006). Construction costs are the focus of the present article.

After a suitable corridor is selected, the road design optimization problem is usually split into three sub-problems: Horizontal Alignment optimization (HA), Vertical Alignment optimization (VA) and EarthWork optimization (EW). HA finds the best feasible sets of curves and tangential line segments between two given points such that the vertical alignment cost is minimized and all constraints are satisfied. (The main HA constraints are the minimum radii of curvature of the circular arcs, which are essential for safety requirements, see Jha et al. 2006.) Given a horizontal alignment, VA determines the best vertical position for the road to minimize construction cost while following design constraints (rate of curvature and bounded grades, see Hancock and Wright 2013). Given a horizontal and vertical alignment, EW minimizes the overall cost of hauling, excavation and embankment.

Optimizing the horizontal alignment in road design is a challenging task as the problem is extremely complicated and intertwined with earthwork and vertical alignment optimization

CONTACT Yves Lucet yves.lucet@ubc.ca

© 2019 Informa UK Limited, trading as Taylor & Francis Group

problems. Some heuristic algorithms have been shown to provide better alignments (Aruga 2005; Tat and Tao 2003) but such methods cannot guarantee even local optimality.

Focusing only on construction costs, Mondal, Lucet, and Hare (2015) studied two derivative-free optimization solvers, namely NOMAD (Nonlinear Optimization with the MADS algorithm, see Le Digabel 2011) and HOPSPACK (Hybrid Optimization Parallel Search PACKage, see Plantenga 2009). While their model guarantees local optimality, it is computationally expensive. One approach to improve computation time is to use a cheaper surrogate function (Audet and Hare 2017), an idea that was already proposed for HA (Pushak 2015). Evaluating the design of the horizontal alignment is extremely expensive in terms of both time and memory consumption. In this article, adaptive multi-fidelity is used to solve HA with low precision first, and as the algorithm finds better horizontal alignments, surrogates with increased accuracy are computed.

Multi-fidelity algorithms have been used in various fields (Fernández-Godino et al. 2016), ranging from aerodynamic designs (Huang, Gao, and Zhang 2013) to mechanical engineering designs (Huang et al. 2015) and engineering design (Hu et al. 2018). Multi-fidelity algorithms can be classified into three strategies: adaptation, filtering and fusion (Peherstorfer, Willcox, and Gunzburger 2016). The adaptation strategy refers to adapting the low-fidelity model to the problem at hand using information gathered from the high-fidelity model, *e.g.* using a Kriging model as the low-fidelity model to improve model accuracy (Jones 2001). The fusion strategy refers to utilizing low- and high-fidelity models simultaneously, *e.g.* using the co-Kriging method where the output of the low-fidelity Kriging method is combined with the output of the high-fidelity model at each iteration of the algorithm. One of the benefits of this technique is that no *a priori* information about the problem is required (Park, Haftka, and Kim 2015). Lastly, methods based on filtering employ high fidelity when more precision is required (Fernández-Godino et al. 2016). This technique usually starts with a low-accuracy surrogate model wherein the objective is to minimize the surrogate model. As the algorithm converges to better points in the search space, the surrogate model utilizes its higher accuracy. Examples of this technique can be found in Takaki and Yamashita (2011), Polak and Wetter (2006) and Wetter and Polak (2004).

There are many techniques for converting high-fidelity models to low-fidelity ones, including dimensionality reduction, linearization, less refinement, simpler geometry, simpler physics, *etc.* (Fernández-Godino et al. 2016). In this article, in order to generate the low-fidelity model for the objective function, a less refined technique is applied wherein fewer '*cross sections*' are used for optimizing the vertical alignment for a fixed horizontal alignment. Both Generalized Pattern Search (GPS) with adaptive control (Polak and Wetter 2006) and the trust-region algorithm with controlled error (Takaki and Yamashita 2011) are used to solve the horizontal problem in road design. Polak and Wetter (2006) proposed a new version of GPS that adaptively controls the error of the objective function. The algorithm begins with the low-fidelity version of the true-objective function. Then, as it converges to better points in the search space, a higher-fidelity model is employed. Even though the objective function is not accurate, the algorithm still converges to a local minimizer, and solve the problem faster than using the algorithm with only the high-fidelity model. Alternatively, using a trust-region algorithm based on the support vector regression model (Takaki and Yamashita 2011), the optimization problem is solved faster than using only a surrogate-free version, and the algorithm converges to a stationary point.

The algorithms chosen fall within multi-fidelity hierarchical methods as they do not employ high-fidelity models at each iteration, and they use high-fidelity models when they are close enough to the stationary point. They can also be considered as filtering multi-fidelity algorithms as the model management strategy of the algorithm decides when either the high-fidelity model or low-fidelity model should be used.

The remainder of the article is structured as follows. In Section 2, multi-fidelity algorithms are recalled, and applied to HA in Section 3. Section 4 reports results and comparisons on 35 road problems; the multi-fidelity framework provides significant computation time savings. Section 5 concludes the article with the consideration of possible future work.

2. Multi-fidelity algorithms

Multi-fidelity algorithms are discussed in this section, and applied to HA in the following section. Definitions from Audet and Hare (2017) are adopted here.

The box-constrained optimization problem

$$\min_x \{f(x) : x \in R^d,\ l_i \le x_i \le u_i,\ i = 1, \dots, d\},$$

is considered, where $f : \mathbb{R}^d \to \mathbb{R}$, and l, u are the lower and upper bounds, respectively. A *box-constrained optimization problem with error*

$$\min_x \{\tilde{f}(x, \epsilon) : x \in R^d,\ \epsilon \in \mathbb{R}_+,\ l_i \le x_i \le u_i\}$$

is also considered, where $\tilde{f} : \mathbb{R}^d \times \mathbb{R}_+ \to \mathbb{R}$, $x \in \mathbb{R}^d$ and $\epsilon \in \mathbb{R}_+$.

A set of the non-negative linear combination of vectors in Ω is said to be a *positive spanning set* if, and only if, $\mathrm{PSpan}(\Omega) = \mathbb{R}^d$, where

$$\mathrm{PSpan}(\Omega) \triangleq \left\{ \sum_{i=1}^{m} \lambda_i \theta_i : m \ge 1,\ \lambda_i \ge 0,\ \theta_i \in \Omega \right\}.$$

A set of vectors Ω is said to be *positive linearly independent* if, and only if, for any $\theta \in \Omega$, $\theta \notin \mathrm{PSpan}(\Omega \setminus \{\theta\})$. A set is said to be a *positive basis* if, and only if, it is positive linearly independent and a positive spanning of $\mathbb{R}^d$.

The *canonical maximal positive basis* of $\mathbb{R}^d$ is

$$\Omega_{\max} = \{e_1, e_2, \dots, e_d, -e_1, \dots, -e_d\},$$

where e_i is the ith unit vector, while the *canonical minimal positive basis* of $\mathbb{R}^d$ is

$$\Omega_{\min} = \left\{ e_1, e_2, \dots, e_d, -\left(\sum_{i=1}^{d} e_i \right) / d \right\}.$$

A *Mesh* M is a set of vectors defined as

$$M(x, \delta) = \{x + \delta D y : y \in \mathbb{N}^p\} \subset \mathbb{R}^d,$$

where $x \in \mathbb{R}^d$, $\delta \in \mathbb{R}_+ = \{x \in \mathbb{R} : x \ge 0\}$, $D = GZ \in \mathbb{R}^{d \times p}$, $G \in \mathbb{R}^{d,d}$ is an invertible matrix, $Z \in \mathbb{Z}^{d \times p}$, and the columns of Z form a positive spanning set for $\mathbb{R}^d$.

2.1. Generalized pattern search algorithms with adaptive precision control

Generalized Pattern Search (GPS) can be applied to both unconstrained and constrained optimization problems (Polak and Wetter 2006). It is a direct search method that relies on a poll step. If the set of polling directions is chosen properly, the algorithm converges to a local minimizer. A good poll set must satisfy three conditions: it should have only a few vectors, it should cover every half-space of $\mathbb{R}^d$, and all vectors in the poll set should be of similar size (Audet and Hare 2017). When the poll set is a positive basis, the first two conditions are satisfied. Using a positive basis ensures that there exists at least one vector in the set that is a descent direction (Audet and Hare 2017, Theorem 6.5).

The GPS for unconstrained optimization problems with adaptive precision (Polak and Wetter 2006) starts with an objective function having low accuracy (a low-fidelity model). As the algorithm converges, it employs a smaller size for the mesh size parameter, and a higher-fidelity model

is used. It has been proven that, when a proper poll step (positive basis) is selected and the error is dependent on the mesh size parameter, the algorithm converges to a stationary point (Polak and Wetter 2006).

The algorithm considers three conditions that make it different from the general version of the GPS algorithm. First, it assumes that the objective function is not precise and contains error. Second, the objective function error can be controlled. Third, the more computation time spent on the objective function, the more accuracy is gained. The main objective of this algorithm is to control the error adaptively so that the algorithm converges to a stationary point.

The pseudocode for the GPS algorithm with adaptive precision control is presented as Algorithm 1. To control the error, the algorithm defines a new parameter called ϵ_k that determines the precision of the objective function. One other important difference between Algorithm 1 and the general version of GPS is that Algorithm 1 does not increase δ_{k+1} at successful iterations.

The description of the algorithm is as follows. In *Step 0*, the initial error of the objective function, ϵ_0, is initialized with a large value since initial points are assumed to be far away from a local optimizer.

In *Step 1*, similar to the general framework of GPS, the algorithm carries out the search process on E_k, which is a subset of the Mesh M_k. However, in contrast to the GPS algorithm, this algorithm evaluates the points in E_k using the surrogate function. (In this study, a search step is not employed as it has not shown positive results in the outcome in preliminary studies.)

Algorithm 1 GPS with adaptive precision control (GPS$_\epsilon$)

Input: $f : \mathbb{R}^d \to \mathbb{R}$, starting point x_0.

Step 0—Initialization:

$\epsilon_0 \in (0, 1)$	▷ the initial error value,
$\delta_0 \in (0, \infty)$	▷ an initial mesh size parameter,
$D = GZ$	▷ a positive spanning matrix,
$Z \in \mathbb{Z}^{d \times p}$	▷ a $d \times p$ matrix of integer numbers,
$\tau_{\text{inc}} > 1$	▷ a mesh size increasing parameter,
$\tau_{\text{dec}} \in (0,\ 1)$	▷ a mesh size decreasing parameter,
$k = 0$	▷ an iteration counter,
$\epsilon_{\text{stop}} \in\ (0, \infty)$	▷ a stopping tolerance,
$E \in \mathbb{N}$	▷ an error control parameter.

Step 1—Global search. Construct the global search set E_k as a finite subset of the mesh M_k:

if $\tilde{f}(x, \epsilon_k) - \tilde{f}(x_k, \epsilon_k) < -\zeta\ \epsilon_k$ for at least one $x \in E_k$, **then**

$x_{k+1} = x$, $\delta_{k+1} = \delta_k$, $\epsilon_{k+1} = \epsilon_k$ and go to Step 3.

else go to Step 2.

Step 2—Poll. Select a positive spanning set $\iota_k \subseteq D$:

if $\tilde{f}(x, \epsilon_k) - \tilde{f}(x_k, \epsilon_k) < -\zeta\ \epsilon_k$ for at least one $x \in \iota_k$, **then**

$x_{k+1} = x$, $\delta_{k+1} = \delta_k$, $\epsilon_{k+1} = \epsilon_k$.

else

$x_{k+1} = x_k$, $\delta_{k+1} = \tau_{\text{dec}}\, \delta_k$.

if E sequential poll failures have occurred, **then**

$\epsilon_{k+1} = \tau_{\text{dec}}\, \epsilon_k$.

else

$\epsilon_{k+1} = \epsilon_k$.

Step 3—Stop:

if $\delta_k \le \epsilon_{\text{stop}}$, **then** stop.

else $k = k + 1$ and go to Step 1.

In *Step 2*, a subset of D is first selected as a local search set. Then a poll step is performed on the selected local set (ι_k). To select ι_k, Polak and Wetter (2006) suggested using a canonical maximal positive basis. In this study, this technique is used to generate polling points at each iteration. If at least one point can be found in ι_k that is significantly better than the incumbent solution, then the incumbent solution is replaced with that point. The reason that the phrase 'significantly better' is used is that, since the difference between the incumbent solution and x should be less than $-\zeta\epsilon_k$ and $\zeta \geq 0$, x should be better than the current solution by a factor of ζ. If no such point can be found, δ_{k+1} is updated by τ_{dec}. Furthermore, if the algorithm cannot find any better points after E sequential iterations, the error ϵ_{k+1} is updated by τ_{dec}. It is worth noting that both δ_k and ϵ_k are updated when a poll failure occurs. So by linking the δ_k and ϵ_k updates, stopping only occurs when the error is small. It has been shown that, using this idea, the algorithm converges to a local minimizer (Polak and Wetter 2006).

In *Step 3*, similar to GPS, the algorithm terminates if δ_k is small enough. Otherwise, it goes to Step 1.

The algorithm converges to a stationary point as long as the cost function f is locally Lipschitz (Polak and Wetter 2006). Even if f is approximated by a family of discontinuous surrogate functions, the algorithm eventually converges to stationary points of f.

2.2. A trust-region algorithm for unconstrained optimization problems with controlled error

Model-based Derivative-Free Optimizations (DFOs) build a surrogate model function from the sample points and its objective function values. They work well when the objective function is differentiable and the model created is easy to minimize (Audet and Hare 2017). The Trust-Region algorithm (TR) for unconstrained optimization problems with controlled error (Takaki and Yamashita 2011) is presented as Algorithm 2.

Algorithm 2 considers pointwise error ϵ_k in the objective function, where ϵ_k is adjusted according to the trust-region size parameter Δ_k. It also considers a penalty parameter C_k for every point in the sample set S. This penalty parameter C_k comes into play when the surrogate model is constructed from the sample set. In particular, sample points with better performance have fewer penalty values, thereby having more impact on the construction of the surrogate model. The algorithm does not only consider x_l in S to construct surrogate model $m_k(\cdot)$ but also penalty and error parameters. Finally, for every point x_k in the sample set, four corresponding values $(x_k, y_k, \epsilon_k, C_k)$ are stored.

In *Step 0*, the model quality control parameters (η_0, η_1) are initialized to (0.1, 0.75), the trust-region increasing and decreasing parameters $\gamma_{\text{inc}}, \gamma_{\text{dec}}$ to 0.5 and 2, the iteration counter k to 0, the minimum size of the sample set N_l to $d+1$ and the initial solution x_0 to the best point in the sample set. The initial error parameter ϵ_0 is set to 0.1.

The next sub-step is to construct the sample set S. Similar to the general framework of the trust-region algorithm, the sample points in S can be created using a random method (Takaki and Yamashita 2011), Latin hypercube sampling (Iman 2008) or a positive basis. However, in contrast to the general version of the trust-region algorithm, wherein only x is stored for each point, in the trust-region algorithm with controlled error, the error and penalty values are also stored for each point. Here, to construct the sample points in S, the canonical minimal positive basis is chosen, since preliminary studies have shown that it would be a good choice. It is worth noting that, for every point in S, ϵ_l and C_l are initialized with ϵ_0 and C_0, respectively.

In *Step 1*, a surrogate model is built using the support vector regression model. Since the objective function is assumed to contain errors, the surrogate model considers pointwise error (ϵ_k) in its design. Moreover, in order to penalize points that are further away from the minimum of the trust region, it considers a penalty parameter C that adaptively adjusts the effect of every point in the sample set. The penalty parameter is updated with

$$C_k = 10^{-\|x_k - x_{k+1}\|/\Delta_k} C_0. \tag{1}$$

Algorithm 2 TR with controlled error

Step 0—Initialization. Choose trust-region parameters:

$0 \le \eta_0 \le \eta_1 < 1$ ▷ model quality control parameters,
$\epsilon_m \in (0, 1)$ ▷ a model control parameter,
$\gamma_{\text{dec}} \in (0, 1)$ ▷ a trust-region decreasing parameter,
$\gamma_{\text{inc}} > 1$ ▷ a trust-region increasing parameter,
ϵ_0 ▷ the initial error parameter,
C_0 ▷ the initial penalty parameter,
construct sample set $S = \{s_1, s_2, \ldots, s_l\}$ where $s_l = (x_l, y_l, e_l, c_l)$,
set $k, \Delta_0, N_l, N_u, x_0$ as in GPS.

Step 1—Construct the model $m_k(x)$ from sample set S:

if$(\max(\|\nabla m_k(x)\|, -\lambda_{\min}(\nabla^2 m_k(x))) < \epsilon_m)$
label the model as unsuccessful, $\rho_k = -1$ and got to Step 4

Step 2—Find the approximate minimum of the model:

compute $x_{k^*} \in \arg\min_{x_k \in B_k} m_k(x)$
evaluate $y_{k^*} = \tilde{f}(x_{k^*}, \epsilon_{k^*})$ with $\epsilon_{k^*} = \epsilon_k$

Step 3—Evaluate the quality of the model:

compute the ratio: $\rho_k = \frac{y_k - y_{k^*}}{m_k(x_k) - m_k(x_{k^*})}$

Step 4—Update the trust-region radius:

successful step: **if** $\rho_k \ge \eta_1$, **then** $\Delta_{k+1} = \gamma_{\text{inc}}\, \Delta_k$.
unsuccessful step: **if** $\rho_k \le \eta_0$, **then** $\Delta_{k+1} = \gamma_{\text{dec}}\, \Delta_k$
else $\Delta_{k+1} = \Delta_k$

Step 5—Adjust the accuracy ϵ_k and penalty parameter C_k:

$\epsilon_{k+1} = o(\Delta_k)$, $C_{k+1} = C_0/\epsilon_{k+1}$

Step 6—Find the best solution:

$x_{k+1} \in \arg\min_{x_j \in (S \cup x_k^*)} (\tilde{f}(x_j, \epsilon_k))$

Step 7—Update process:

if $|S| < N_u$, **then**
insert $(x_{k^*}, y_{k^*}, \epsilon_{k^*}, C_k)$ into S.
else
replace $(x_{k^*}, y_{k^*}, \epsilon_{k^*}, C_k)$ with the worst point in S.
remove any $x \in X$ such that $\|x - x_{k+1}\| > \Delta_k$.
while$(|X| < N_l)$
generate a point inside of the trust region
update the penalty parameter $C_l \quad \forall\, C_l \in S$

Step 8—Stop:

if Δ_k or the gradient of the model is small, **then** stop.
else $k = k + 1$ and go to Step 1.

After building the model, the success of the model is evaluated. If

$$\max\left(\|\nabla m_k(x)\|, -\lambda_{\min}(\nabla^2 m_k(x))\right) < \epsilon_m,$$

the model is labelled as unsuccessful and the algorithm proceeds to Step 4, *i.e.* if the model is almost a constant function, the algorithm does not accept it, labels it as unsuccessful, and shrinks the trust-region radius in Step 4 ($\lambda_{\min}$ denotes the minimum eigenvalue).

In *Step 2*, the approximate minimum of the model is found. Since the algorithm searched for the approximate minimum of the model within the hypersphere without any external constraints, a simple optimization algorithm is used. In this study, the truncated conjugate gradient (Conn, Gould, and Toint 2000, Section 7.5.1) is implemented as it provides the approximate minimum of the model quickly with good accuracy.

In *Step 3*, the quality of the model is evaluated using the objective function value with ϵ_k (the error value at the current iteration).

In *Step 4*, if $\rho_k > \eta_1$ (accurate model), the current trust-region radius Δ_{k+1} is expanded. Otherwise, the trust-region radius is shrunk.

In *Step 5*, the error is updated based on the trust-region radius, Δ_k. Because the derivative of the objective function is not available, the distance between a point x to the solution cannot be computed. Recall that when x_k is generated by a DFO algorithm with an exact objective function, $k \to \infty$, and $\Delta_k \to 0$ then $\|\nabla f(x_k)\| \to 0$ too (Conn, Scheinberg, and Toint 1997). For this reason, it is a good idea to use Δ_k (the trust-region radius) for the measure. For updating the penalty parameter, C_0/ϵ_{k+1} is used. This formula makes sure that the penalty parameter becomes larger for x_{k^*} with higher accuracy and vice versa.

In *Step 6*, the best solution among the sample points in S and the approximate minimum of the model x_k^* are found and assigned to x_{k+1}.

In *Step 7* If the size of the sample set is less than the maximum size N_u, then x_k^* and all its corresponding values $(y_{k^*},\ \epsilon_{k^*},\ C_k)$ are inserted into the sample set S. Otherwise, x_k^* and its corresponding values take the place of the worst point in S.

For updating the sample set, first the distances between all points in the sample set to x_{k+1} (the best point) are measured. Then if the number of sample points inside the new trust region is less than $d+1$, the furthest points from x_{k+1} will be replaced with newly generated sample points. Note that the new sample points must be inside the new trust regions to be accepted.

When N is less than N_u, it is likely that some points are outside of the trust region, thereby degrading the quality of the model. To surmount this, the penalty of these points is reduced, making them have less effect on the model by reducing C_i. So for any C_k corresponding to $x_k \in S$ and $\|x_k - x_{k+1}\| > \Delta_{k+1}$, C_k is updated using (1).

The construction of a surrogate model using a support vector regression model (Takaki and Yamashita 2011) is briefly recalled here. Note $f : \mathbb{R}^d \to \mathbb{R}$, the true objective function. The goal here is to find a quadratic surrogate model that approximates the behaviour of f. The model $m : \mathbb{R}^d \to \mathbb{R}$ is defined as $m(x) = w^\top \phi(x)$, where $w \in \mathbb{R}^N$ is a coefficient vector, $N = (1/2)(d+1)(d+2)$, d is the dimension of the problem and $\phi(x)$ is a mapping function defined as

$$\phi(x) = [1, x_1, x_2, \ldots, x_n, x_1^2, x_1x_2, \ldots, x_{n-1}x_n, x_n^2]^\top \in \mathbb{R}^N.$$

The value $w = P^\top(\alpha^+ - \alpha^-)$, where P is computed as

$$P = [\phi(x_1), \phi(x_2), \ldots, \phi(x_l)] \in \mathbb{R}^{l \times N},$$

and α^+, α^- are computed by finding an approximate minimizer of the quadratic programming problem,

$$\arg\min_{\alpha^+,\alpha^-} \frac{1}{2}\begin{pmatrix}\alpha^+\\ \alpha^-\end{pmatrix}^\top \begin{pmatrix} K & -K\\ -K & K\end{pmatrix}\begin{pmatrix}\alpha^+\\ \alpha^-\end{pmatrix} + \begin{pmatrix}\epsilon - y\\ \epsilon + y\end{pmatrix}^\top \begin{pmatrix}\alpha^+\\ \alpha^-\end{pmatrix},$$

subject to $0 \le \alpha^+, \alpha^- \le C$ and $\sum_{i=1}^{l}(\alpha_i^+ - \alpha_i^-) = 0$; where $K = PP^\top$, y is a vector of the function values of the sample points $(y_i = \tilde{f}(x_i, \epsilon_i))$ where $i = 1, \ldots, l)$, $\epsilon = [\epsilon_1, \epsilon_2, \ldots, \epsilon_l]$ and $C = [C_1, C_2, \ldots, C_l]$.

3. Multi-fidelity algorithms for solving the HA

Given a corridor with a starting and an ending point, HA computes the parameters of a circular-linear curve that models the road from a satellite viewpoint. An initial alignment is provided and called the baseline. The horizontal alignment is fully defined as a sequence of intersection points (IPs) and radii. Figure 1 shows a corridor with five IPs represented by stars. There are bound constraints drawn as boxes around each IP. The dots represent the input data points providing information on the ground. The centreline is shown as a dashed line, while an optimal alignment is showed as the full curve. Note that the size of HA is proportional to the number of IPs. The technical details of the model are available in Mondal, Lucet, and Hare (2015).

Given a horizontal alignment, the vertical alignment is a planar curve with points (d, z), where d is the distance along the curve from the start of the road, and z is the altitude at that point. The vertical alignment is modelled as a quadratic spline, wherein the road is split into intervals called segments on which the curve is quadratic (or linear). Each segment is further split into intervals called sections. The section endpoints along with associated data are called stations. Each station is provided with a table containing the amount of material needed to build the road above ground, or how much material would be extracted if one were to dig at that point. The size of VA is proportional to the number of stations. Details on this model are available in Beiranvand et al. (2017).

The objective function for HA is the value of VA, *i.e.* the minimum material extraction, loading, moving and unloading costs. Here, the VA problem is considered as a black-box optimization problem, wherein the input parameters are IPs and the station sets, and the output is the optimized VA. Figure 2 shows the box diagram of the objective function of the VA problem. (If the horizontal alignment generated from the IP set x_k is outside of the corridor, or any other safety design constraints are violated, the VA problem will return infinity as the output.)

In Hare, Jaberipour, and Lucet (2018), the authors proposed two surrogate models to approximate the true objective function. Both surrogate models are based on the idea of the reduction of the size of a mixed-integer program needed to tackle the vertical alignment optimization problem at the expense of decreasing the accuracy of the final earthwork cost. In the following, only the UNIform merging technique (UNI) is used, since it has shown better performance.

UNI removes some stations from the station set uniformly in the hope of reducing the computation time. Assume there is a road with $n+1$ stations and with the station set $S = \{s_0, s_1, \ldots, s_n\}$. The method reduces the station set to

$$S_N = \{s_0, s_N, s_{2N}, \ldots, s_{\lfloor (n-1)/N \rfloor N}, s_{\lfloor n/N \rfloor N}\},$$

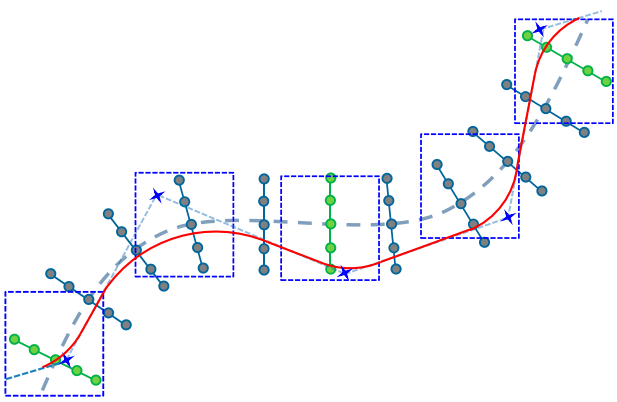

Figure 1. A corridor with five IPs represented by stars (Mondal 2014) (© Sukanto Mondal, 2014, reproduced with the permission of the author).

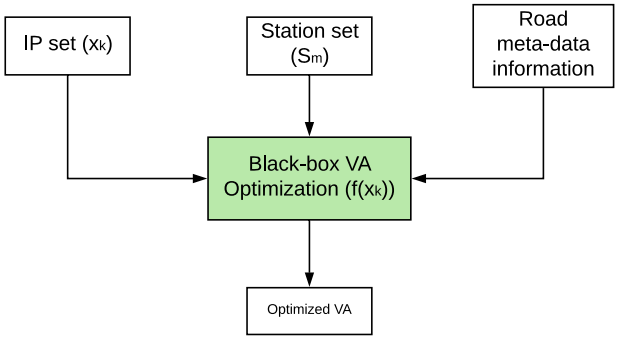

Figure 2. The objective function.

where N varies between $N = 1$, for no reduction, to $N = n$, for reducing the road to two stations. (Control points—stations selected by engineers at which the road is required to have a fixed altitude—are always part of the reduced set of stations; there is also a cap on the maximum length of any segment.) The uniform merging method decreases the number of stations by a factor of $\lfloor (n-1)/N \rfloor + 1$.

In Hare, Jaberipour, and Lucet (2018), the authors evaluated the efficiency of the UNI surrogate on 40 test roads. These sample roads have a minimum 762 stations and a maximum of 11,272 stations. The average distance between stations in the original data is 1 m, making the roads between 0.5 and 12 km long. For the purpose of comparison, each road was solved using all stations. The optimal cost of this base problem is considered as correct and denoted by C_b. The time required to solve the base problem (in seconds) is denoted T_b. Each test road was then solved using the UNI surrogate model, for a variety of surrogate parameters ($N \in \{2, 3, \ldots, 20\}$).

The cost ratio and time ratio are used to evaluate accuracy and efficiency. The cost ratio is C_N/C_b, where C_N is the construction cost computed with the ground approximation provided by a merging technique; it represents a relative construction cost error due to merging. The time ratio is T_N/T_b, where T_N is the computational time for solving the problem with the ground approximation; it represents the relative time reduction due to merging. The section reduction ratio S_N/S_b is introduced, where S_b and S_N are the number of sections in the base road and after merging.

It is immediately clear, and intuitively obvious, that the more stations omitted the greater the time savings. In Figure 3, a quadratic curve gives a coefficient of determination R^2 value greater than 0.99, providing strong evidence that the time ratio is reduced quadratically in relation to the portion of stations omitted.

To use the surrogate model, it is important to have some ability to predict bounds on the accuracy of the output. In this study, quantile regression is used to predict the behaviour of the accuracy. Quantile regression is performed using `ggplot2` in `R`.[1] By considering the 5th, 10th, 25th, 50th, 75th, 90th and 95th percentiles, a predictive model for the UNI surrogate is created. These quantile regression curves are visualized in Figure 4.

Quantile regression curves in Figure 4(a) behave quite strangely. This is due to an overloading of data in the leftmost portion of the graph where the 'percentage of used sections' is less than 5% (corresponding to $N \in \{11, 12, \ldots, 50\}$) and an underrepresentation of data in the rightmost portion of the graph where the 'percentage of used sections' is greater than 50% (corresponding to $N = 1$). If the data are restricted to analyse only when the 'percentage of used sections' is greater or equal to 5%, then the result is considerably cleaner, see Figure 4(b). The predicted model appears quite accurate even with the percentage of used sections as low at 5%. At 5% used sections, the cost error is in the range [0.99, 1.07] at the 5 and 95% quantile regression curves. (Although it should be noted that outliers have errors within [0.91, 1.17].)

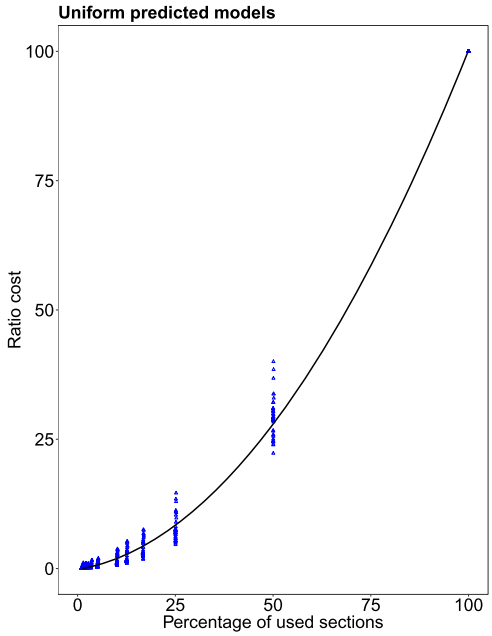

Figure 3. Ratio of time resulting from the UNI surrogate model.

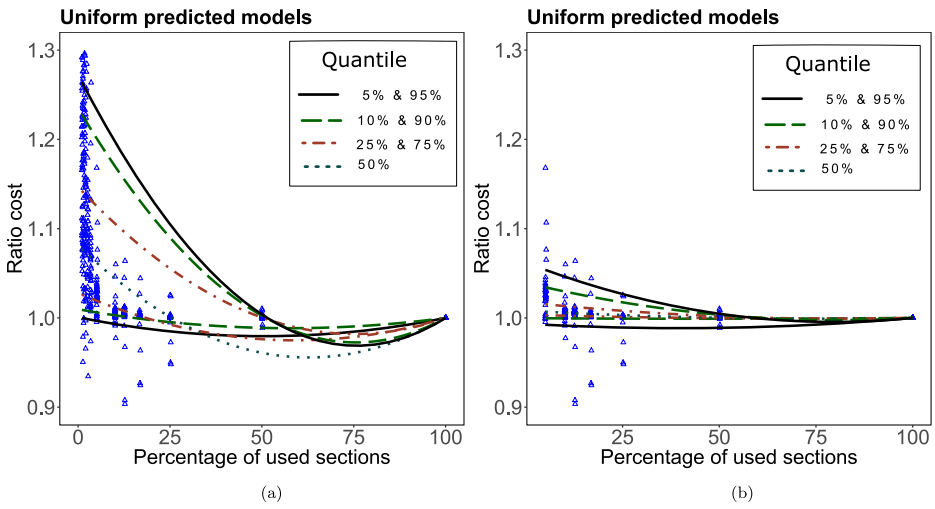

Figure 4. Quantile regression for the cost ratio. (a) Quantile regression using all data. (b) Quantile regression using only a percentage of used sections greater than 5%.

Even though the method is simple, experiments carried out on 40 roads show that it can decrease the solution time by a factor of eight with only a small loss of accuracy (Hare, Jaberipour, and Lucet 2018). In this study, this technique is used as a surrogate model for reducing the station set, and as a result, solving the vertical alignment optimization problem faster.

Table 1 is created from experiments carried out on 40 roads (Hare, Jaberipour, and Lucet 2018). Using that table and given the surrogate model $\tilde{f}$, the error value ϵ_k at iteration k, the IP set x_k, and the station set with no reduction S, N is first computed from ϵ_k and then the uniform merging method is used to create S_N. Merged station sets S_N and meta-data are computed once and stored to save

Table 1. A mapping table from ϵ_k to N.

Error (ϵ_k)	N
$\epsilon_k > 0.12$	20
$0.06 \leq \epsilon_k \leq 0.12$	10
$0.034 \leq \epsilon_k < 0.06$	6
$0.01 \leq \epsilon_k < 0.034$	4
$0 < \epsilon_k < 0.01$	2
$\epsilon_k = 0$	1

Table 2. Description of 35 different roads.

Road no.	No. of IPs	No. of stations	Road no.	No. of IPs	No. of stations
1	2	49	19	17	252
2	1	33	20	19	368
3	5	107	21	8	523
4	3	114	22	2	470
5	3	114	23	3	97
6	2	116	24	3	95
7	2	120	25	2	98
8	2	120	26	2	11
9	7	178	27	2	952
10	2	120	28	5	1,164
11	2	129	29	2	2,288
12	8	303	30	15	446
13	4	184	31	17	367
14	2	222	32	12	9,343
15	13	203	33	19	1,168
16	2	227	34	9	941
17	2	228	35	1	24
18	2	234	–	–	–

computation time. At each iteration, the algorithm obtains the pre-computed set S_N and solves VA to obtain x_k.

4. Experimental results

In this section, the effect of the multi-fidelity framework is studied for the two derivative-free optimization algorithms on 35 different roads provided by the industrial partner (Softree Technical System Inc.—abbreviated as Softree). Multi-fidelity is compared against single-fidelity within a single algorithm style.

The information gathered contains the initial alignment (X- and Y-coordinates and radius of IPs), the station set and cut/fill material information. Table 2 provides a summary of each road, including the number of IPs and the number of stations. Most roads contain one to ten IPs. Since the radii of IPs are fixed, the number of IPs is exactly one half the number of variables in the resulting HA problem, e.g. Road 1 with two IPs contains four variables. Even with this small number of variables, solving HA can take more than 20 hours with several roads reaching the 24 hour time limit.

The algorithms are run on three different workstation PCs using 64 bit Windows® 7. The first computer has an Intel® Core™ i7, 2.8 GHz CPU with 16 GB RAM, and is used to run the HA algorithms on small roads. The second and third computers are Intel® Xeon® 3.5 GHz CPUs with 64 GB RAM; they are used to run the algorithms on medium to large roads. The algorithms are implemented using MATLAB® 2017 and Visual Studio™ 2013.

4.1. Parameter tuning

Before performing a comparison, experiments are carried out to determine good parameters for the GPS_ϵ and TR_ϵ algorithms. Tuning parameters on a subset of the roads that can be optimized quickly is a compromise between accepting default values that have not been selected with the road design problem in mind, and tuning parameters on all the roads, which is too time consuming. These, and other experiments, make use of performance profiles (Dolan and Moré 2002) that have become the gold-standard in optimization benchmark literature (Beiranvand, Hare, and Lucet 2017).

Parameter tuning is performed to optimize the convergence speed of the two algorithms. The algorithms are tuned on roads 1, 2, 5, 8 and 35. The GPS_ϵ and TR_ϵ algorithms are tuned in their multi-fidelity configuration, with $\epsilon_0 = 0.1$, and in a single-fidelity configuration with $\epsilon_0 = 0$. These two configurations are called GPS_0 and GPS_ϵ, and TR_0 and TR_ϵ, for their respective algorithms. The maximum number of function evaluations F_{eval} is fixed at

$$F_{\text{eval}} = 100\ \min(|IP|^2, 5|IP|), \tag{2}$$

where $|IP|$ is the size of the IP set.

Table 3 presents the different values of parameters for which GPS_0 and GPS_ϵ are tuned. Figure 5 displays the performance profiles of the different configurations of GPS_0, and Figure 6 the performance profiles of GPS_ϵ for the different configurations with a tolerance $\tau = 10\%$ and $\tau = 5\%$.

Examining the performance profiles in Figures 5 and 6, the best parameter selections are determined to be Configuration 4 for GPS_0 and Configuration 49 for GPS_ϵ. These correspond to $\delta_0 = 5, \tau_{\text{dec}} = 0.5$ for GPS_0 and $E = 1, \delta_0 = 5, \tau_{\text{dec}} = 0.5$ for GPS_ϵ.

A similar process is completed for TR_0 and TR_ϵ. However, for the TR_ϵ algorithms, the parameters γ_{inc}, η_0, η_1, ϵ_m and C_0 are set according to Takaki and Yamashita (2011). The parameters tuned, and the values tested, are presented in Table 4.

After analysis of the resulting performance profiles (available on request) it is decided to use $\Delta_0 = 5, \gamma_{\text{dec}} = 0.1$ for TR_0 and $E = 1, \Delta_0 = 5, \gamma_{\text{dec}} = 0.1$ for TR_ϵ.

Table 3. Parameter values for the GPS_0 and GPS_ϵ algorithms.

Parameter	GPS_0	GPS_ϵ
ϵ_0	0	0.1
E	NA (0)	$\{1, 2, , 5, 10\}$
δ_0	$\{5, 2.5, 1.25\}$	$\{5, 2.5, 1.25\}$
τ_{dec}	$\{0.01, 0.1, 0.3, 0.5, 0.7, 0.9\}$	$\{0.01, 0.1, 0.3, 0.5, 0.7, 0.9\}$
τ_{inc}	$\tau_{\text{inc}} = 1/\tau_{\text{dec}}$	$\tau_{\text{inc}} = 1/\tau_{\text{dec}}$

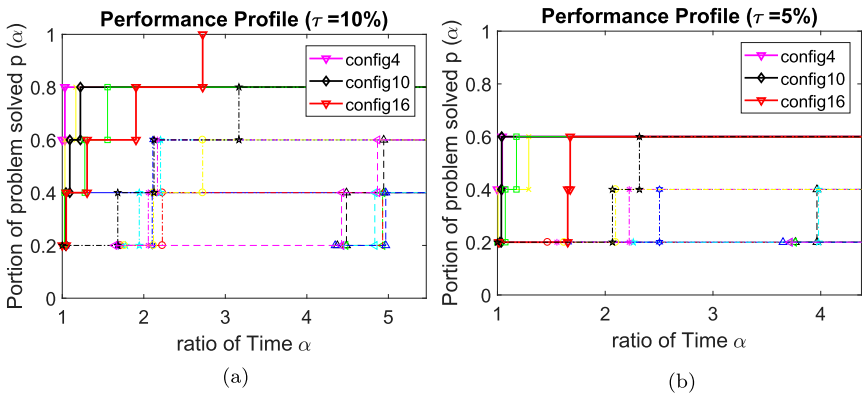

Figure 5. The performance profile of the GPS_0 algorithm for 18 different configurations on five small roads. (a) Different configurations of GPS_0 with 10% tolerance. (b) Different configurations of GPS_0 with 5% tolerance.

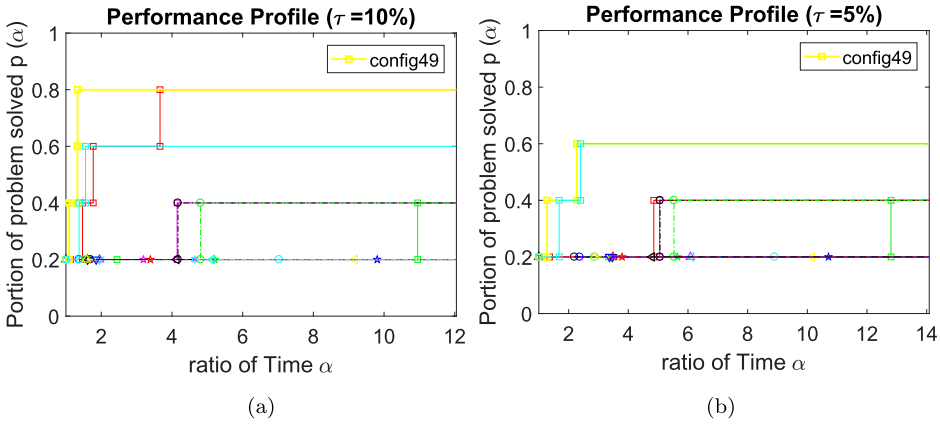

Figure 6. The performance profile of the GPS$_\epsilon$ algorithm for 72 different configurations on five small roads. (a) Different configurations of GPS$_\epsilon$ with 10% tolerance. (b) Different configurations of GPS$_\epsilon$ with 5% tolerance.

Table 4. Parameter values for the TR$_0$ and TR$_\epsilon$ algorithms.

Parameter	TR$_0$	TR$_\epsilon$
ϵ_0	0	0.1
E	NA (0)	{1, 2, , 5, 10}
Δ_0	{5, 2.5, 1.25}	{5, 2.5, 1.25}
γ_{dec}	{0.1, 0.3, 0.5, 0.7, 0.9}	{0.1, 0.3, 0.5, 0.7, 0.9}

Table 5. Speed-up and cost difference for the GPS algorithms.

Speed factor	No. of roads	Cost difference (%)	No. of roads
< 0.7	1	< 0	3
0.7 to 0.9	1	0.0 to 1.0	17
0.9 to 1.1	1	1.0 to 2.0	4
1.1 to 2.0	7	2.0 to 5.0	1
2.0 to 3.0	9	> 5.0	0
> 3	6		
	All roads		All roads
Average	2.52	Average	0.41%

4.2. Multi-fidelity versus single-fidelity

With the parameters tuned, testing is next run on all 35 test roads presented in Table 2. An analysis of the speed-up resulting through the use of multi-fidelity surrogates and the accuracy resulting from the multi-fidelity surrogate algorithms (Tables 5 and 6) is performed. The speed-up factor is calculated by dividing the computation time of the multi-fidelity algorithm by the computation time of the single-fidelity one. The average speed-up is the sum of the speed-up factors divided by the number of test roads. The cost difference is

$$\frac{f_\epsilon - f_0}{f_b},$$

where f_ϵ is the cost given by the multi-fidelity algorithm, f_0 the cost given by the single-fidelity algorithm, and f_b the cost at the baseline (initial alignment). The average cost difference is the sum of the cost differences divided by the number of test roads.

Table 6. Speed-up and cost difference for the TRSVR algorithms.

Speed factor	No. of roads	Cost difference	No. of roads
< 0.7	1	< 0	2
0.7 to 0.9	1	0.0 to 1.0	27
0.9 to 1.1	6	1.0 to 2.0	3
1.1 to 2.0	26	2.0 to 5.0	2
2.0 to 3.0	0	> 5	0
> 3	0		
	All roads		All roads
Average	1.31	Average	0.40%

Table 5 presents the results of the speed-up factor and cost difference between GPS_0 and GPS_ϵ. Table 6 provides the same information for TR_0 and TR_ϵ.

It can be seen that, on average, using the multi-fidelity framework improves the speed of the GPS algorithms by 152% while keeping the cost difference at 0.41%. Similarly, TR_ϵ improves the speed of the TR_0 algorithm by 31% while the cost difference is 0.40% on average. Overall, this suggests that the use of multi-fidelity surrogates presents a very powerful opportunity for improving solve time of the HA problem.

It should be noted that the number of roads in Tables 5 adds up to 23. This is due to the fact that GPS_0 and GPS_ϵ are unable to solve 12 of the roads within the allotted time constraint. Conversely, TR_0 is able to solve 34 out of 35 test problems and TR_ϵ is able to solve all 35 test problems. This suggests that TR_ϵ is a more appropriate choice for use of multi-fidelity surrogates to solve the HA problem.

The size of the road is found to affect the speed-up for the GPS algorithms significantly. When the size of the HA problem is large (three or more IPs), multi-fidelity does not help the algorithm in the case of only one road, and there is no road that causes the algorithm to take more computation time; on all other roads, a speed-up was observed. It is also noted that, as the number of IPs increases, the speed-up increases and the cost difference decreases on average. On the contrary, it is observed that the number of IPs has very little effect on the efficiency of the TR algorithms.

5. Conclusion

Solving HA can take hours using Derivative-Free Optimization (DFO) algorithms. To speed up computation, a GPS and a TR multi-fidelity algorithm were run on 35 different roads. The experimental results showed that the multi-fidelity framework increases the speed of GPS_0 by an average of 152% (31% for TR_0) with a negligible impact on accuracy. (When design costs are dominated by bridges and tunnels, the speed-up from multi-fidelity algorithms still occurs but may represent a smaller portion of the overall computational costs.)

In additional tests, it was found that, when the size of the HA problem is small (small number of IPs), the multi-fidelity framework does not help and even degrades the speed of the algorithm. However, as the HA problem becomes larger, the multi-fidelity framework provides speed improvement. This is helpful, since good HA algorithms to optimize small roads are already available.

Even though the multi-fidelity framework helps the GPS_0 and TR_0 algorithms to improve their speed, there is still room for further improvement. In particular, the tested software is a basic implementation based exactly on the pseudocodes presented in Polak and Wetter (2006) and Takaki and Yamashita (2011). The addition of global search heuristics or parallel programming could provide improvement in GPS_ϵ and TR_ϵ (as well as GPS_0 and TR_0). However, a more fruitful approach would probably be to rely on current high-quality software (such as NOMAD) and focus on incorporating the multi-fidelity techniques explored in this article into such software. Another valuable direction for future research would be to study multi-fidelity algorithms that can take advantage of the main

characteristics of the road design problem, in particular the availability of a multi-fidelity surrogate with several levels of precision and having a high precision surrogate even for coarse precisions.

In terms of optimality, both the GPS_ϵ and TR_ϵ studied guarantee local optimality. In the future, anyone interested in global optimality could choose global optimization algorithms having an approach such as branch and cut (Tawarmalani and Sahinidis 2005) to find the global minimizer. Since the road design optimization problem is a non-convex mixed-integer nonlinear programming problem, global optimization solvers can be used such as BARON (Sahinidis 1996) to solve for small size roads, and for large-size roads the solution obtained by a global solver can be used as an initial point for the GPS or the TR algorithm.

Note

1. http://ggplot2.tidyverse.org/reference/geom_quantile.html and https://www.r-project.org/

Acknowledgements

Part of the computation in this research was carried out using a software library provided by Softree Technical System Inc.

Disclosure statement

No potential conflict of interest was reported by the authors.

Funding

This work was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC) under Collaborative Research and Development grant [#CRDPJ 479316-15] sponsored by Softree Technical Systems Inc.; the University of British Columbia under the Graduate Entrance Scholarship and the University Graduate Fellowship; the Canada Foundation for Innovation (CFI) under Leaders Opportunity Fund (LOF, John R. Evans Leaders Fund — Funding for research infrastructure for the Computer-Aided Convex Analysis (CA2) laboratory); and the province of British Columbia under British Columbia Knowledge Development Fund (BCKDF).

ORCID

Warren Hare http://orcid.org/0000-0002-4240-3903
Yves Lucet http://orcid.org/0000-0003-0349-6737

References

Aruga, Kazuhiro. 2005. "Tabu Search Optimization of Horizontal and Vertical Alignments of Forest Roads." *Journal of Forest Research* 10 (4): 275–284.

Audet, Charles, and Warren Hare. 2017. *Derivative-Free and Blackbox Optimization*. Springer Series in Operations Research and Financial Engineering. Cham, Switzerland: Springer International Publishing. doi:10.1007/978-3-319-68913-5.

Beiranvand, Vahid, Warren Hare, and Yves Lucet. 2017. "Best Practices for Comparing Optimization Algorithms." *Optimization and Engineering* 18 (4): 815–848.

Beiranvand, Vahid, Warren Hare, Yves Lucet, and Shahadat Hossain. 2017. "Multi-Haul Quasi Network Flow Model for Vertical Alignment Optimization." *Engineering Optimization* 49 (10): 1777–1795. doi:10.1080/0305215X.2016.1271880.

Conn, Andrew R., Nicholas I. M. Gould, and Philippe L. Toint. 2000. *Trust-Region Methods*. Philadelphia, PA: SIAM. doi:10.1137/1.9780898719857.

Conn, Andrew R., Katya Scheinberg, and Philippe L. Toint. 1997. "On the Convergence of Derivative-Free Methods for Unconstrained Optimization." In *Approximation Theory and Optimization: Tributes to M. J .D. Powell,* edited by M. D. Buhmann and A. Iserles, 83–108. Cambridge, UK: Cambridge University Press. http://assets.cambridge.org/97805215/81905/frontmatter/9780521581905_frontmatter.pdf.

Dolan, Elisabeth D., and Jorge J. Moré. 2002. "Benchmarking Optimization Software with Performance Profiles." *Mathematical Programming* 91: 201–213.

Fernández-Godino, M. Giselle, Chanyoung Park, Nam-Ho Kim, and Raphael T. Haftka. 2016. "Review of Multi-Fidelity Models." Preprint arXiv: 1609.07196.

Hancock, Michael W., and Bud Wright. 2013. "A Policy on Geometric Design of Highways and Streets." Washington, DC: American Association of State Highway and Transportation. https://deldot.gov/Business/drc/manuals/aashto_design_of_streets_and_highways_errata.pdf.

Hare, Warren, Majid Jaberipour, and Yves Lucet. 2018. *Multi-Fidelity Surrogates for Vertical Alignment Optimization in Road Design*. Technical report. Kelowna, Canada: University of British Columbia.

Hu, Jiexiang, Qi Zhou, Ping Jiang, Xinyu Shao, and Tingli Xie. 2018. "An Adaptive Sampling Method for Variable-Fidelity Surrogate Models Using Improved Hierarchical Kriging." *Engineering Optimization* 50 (1): 145–163.

Huang, Likeng, Zhenghong Gao, and Dehu Zhang. 2013. "Research on Multi-Fidelity Aerodynamic Optimization Methods." *Chinese Journal of Aeronautics* 26 (2): 279–286.

Huang, Edward, Jie Xu, Si Zhang, and Chun-Hung Chen. 2015. "Multi-Fidelity Model Integration for Engineering Design." *Procedia Computer Science* 44: 336–344.

Iman, Ronald L. 2008. "Latin Hypercube Sampling." *Wiley StatsRef: Statistics Reference Online*. doi:10.1002/9781118445112.stat03803.

Jha, Manoj Kumar, Paul Schonfeld, Jyh-Cherng Jong, and E. Kim. 2006. *Intelligent Road Design*. Advances in Transport series, Vol. 19. Southampton, UK: WIT Press.

Jones, Donald R. 2001. "A Taxonomy of Global Optimization Methods Based on Response Surfaces." *Journal of Global Optimization* 21 (4): 345–383.

Le Digabel, Sébastien. 2011. "Algorithm 909: NOMAD: Nonlinear Optimization with the MADS Algorithm." *ACM Transactions on Mathematical Software* 37 (4): 1–15.

Mondal, Sukanto. 2014. "Horizontal Alignment Optimization in Road Design." Master's thesis, University of British Columbia. https://open.library.ubc.ca/cIRcle/collections/ubctheses/24/items/1.0074351.

Mondal, Sukanto, Yves Lucet, and Warren Hare. 2015. "Optimizing Horizontal Alignment of Roads in a Specified Corridor." *Computers & Operations Research* 64 (2015): 130–138. doi:10.1016/j.cor.2015.05.018.

Park, Chanyoung, Raphael T. Haftka, and Nam-Ho Kim. 2015. "Experience with Several Multi-Fidelity Surrogate Frameworks." In *11th World Congress on Structural and Multidisciplinary Optimization (WCSMO-11)*, 7–12. University of Sydney: ISSMO. http://web.aeromech.usyd.edu.au/WCSMO2015/papers/1056_paper.pdf.

Peherstorfer, Benjamin, Karen Willcox, and Max Gunzburger. 2016. *Survey of Multifidelity Methods in Uncertainty Propagation, Inference, and Optimization*. ACDL Technical Report TR16-1. Aerospace Computational Design Laboratory, Department of Aeronautics and Astronautics, MIT, Cambridge, MA. http://web.mit.edu/pehersto/www/preprints/multi-fidelity-survey-peherstorfer-willcox-gunzburger.pdf.

Plantenga, Todd D. 2009. *HOPSPACK 2.0 User Manual*. Technical Report SAND2009-xTBD. Sandia National Laboratories, Albuquerque, NM and Livermore, CA.

Polak, Elijah, and Michael Wetter. 2006. "Precision Control for Generalized Pattern Search Algorithms with Adaptive Precision Function Evaluations." *SIAM Journal on Optimization* 16 (3): 650–669. doi:10.2172/813385.

Pushak, Yasha. 2015. "Road Design Optimization with a Surrogate Function." Master's thesis, University of British Columbia.

Sahinidis, Nikolaos V. 1996. "BARON: A General Purpose Global Optimization Software package." *Journal of Global Optimization* 8 (2): 201–205.

Takaki, Jun, and Nobuo Yamashita. 2011. "A Derivative-Free Trust-Region Algorithm for Unconstrained Optimization with Controlled Error." *Numerical Algebra, Control & Optimization* 1 (1): 117–145.

Tat, Chan Weng, and Fan Tao. 2003. "Using GIS and Genetic Algorithm in Highway Alignment Optimization." In *Proceedings of the 2003 IEEE International Conference on Intelligent Transportation Systems*, Vol. 2, 1563–1567. Piscataway, NJ: IEEE. doi:10.1109/ITSC.2003.1252747.

Tawarmalani, Mohit, and Nikolaos V. Sahinidis. 2005. "A Polyhedral Branch-and-Cut Approach to Global Optimization." *Mathematical Programming* 103 (2): 225–249.

Transportation. 2017. "Transportation in Canada 2016." https://www.tc.gc.ca/eng/policy/transportation-canada-2016.html.

Wetter, Michael, and Elijah Polak. 2004. "A Convergent Optimization Method Using Pattern Search Algorithms with Adaptive Precision Simulation." *Building Services Engineering Research and Technology* 25 (4): 327–338.

Appendix. Data tables

Here, the detailed results on 35 roads for the GPS_0, GPS_ϵ, TR_0 and TR_ϵ algorithms, and NOMAD are presented.

It should be noted that in Tables A1, A2, A3 and A4, 'Cost' is the optimized VA cost obtained, 'Time' is the total computation time measured, 'Status' and 'Reason' are the status and reason that an algorithm terminates, 'Memory' is the memory used by a computer to solve the problem, 'True FC' and 'Surrogate FC' are the number of surrogate and true function calls used for the algorithm to solve the HA problem.

In Tables A1 and A2, 'Mesh small' refers to the fact that the GPS algorithm stops as it reaches the final mesh size. Furthermore, '24h T' demonstrates that the algorithm reaches the maximum computation time before finding a local minimizer.

In Tables A3 and A4, 'TR small' means that the algorithm stops because the trust region is small.

Table A1. The detailed results of GPS_0 for the HA problem across all roads.

	GPS_0						
Road number	Cost ($)	Time (s)	Status	Reason	Memory (mB)	True FC (#)	Surrogate FC (#)
1	252,975	334	Success	Mesh small	1,645	106	0
2	44,458	160	Success	Mesh small	1,450	41	0
3	537,873	5,023	Success	Mesh small	1,643	690	0
4	127,411	1,211	Success	Mesh small	1,645	150	0
5	8,554,830	1,054	Success	Mesh small	1,636	137	0
6	10,951,800	762	Success	Mesh small	1,643	90	0
7	5,889,700	1,069	Success	Mesh small	1,649	130	0
8	7,273,130	1,729	Success	Mesh small	1,654	144	0
9	363,861	5,538	Success	Mesh small	1,600	463	0
10	6,986,830	1,598	Success	Mesh small	1,653	94	0
11	6,082,110	1,533	Success	Mesh small	1,651	128	0
12	204,763	17,273	Success	Mesh small	1,702	694	0
13	650,128	4,566	Success	Mesh small	1,193	164	0
14	278,496	1,320	Success	Mesh small	1,275	53	0
15	1,514,740	43,276	Success	Mesh small	1,395	1,732	0
16	356,061	1,242	Success	Mesh small	1,268	49	0
17	244,938	1,767	Success	Mesh small	1,622	56	0
18	2,853,810	3,626	Success	Mesh small	1,632	106	0
19	614,144	68,809	Success	Mesh small	1,608	3,482	0
20	121,374	–	Fail	24h T	–	–	–
21	3,088,040	48,963	Success	Mesh small	1,369	589	0
22	230,578	5,504	Success	Mesh small	1,594	59	0
23	149,631	7,550	Success	Mesh small	1,596	479	0
24	222,118	3,030	Success	Mesh small	1,601	148	0
25	222,550	789	Success	Mesh small	1,441	65	0
26	244,800	195	Success	Mesh small	1,436	86	0
27	377,133	–	Fail	24h T	–	–	–
28	568,715	–	Fail	24h T	–	–	–
29	11,012,600	–	Fail	24h T	–	–	–
30	1,016,780	–	Fail	24h T	–	–	–
31	1,075,830	–	Fail	24h T	–	–	–
32	47,097,000	–	Fail	24h T	–	–	–
33	30,894,600	–	Fail	24h T	–	–	–
34	30,155,800	–	Fail	24h T	–	–	–
35	28,887	73	Success	Mesh small	1,665	35	0

Table A2. Detailed results of GPS_ϵ for the HA problem across all roads.

	GPS_ϵ						
Road number	Cost ($)	Time (s)	Status	Reason	Memory (mB)	True FC (#)	Surrogate FC (#)
1	251,160	226	Success	Mesh small	1,298	21	169
2	44,752	39	Success	Mesh small	1,284	10	41
3	543,235	1,504	Success	Mesh small	1,288	33	661
4	127,346	563	Success	Mesh small	1,284	21	253
5	8,604,670	613	Success	Mesh small	1,285	24	289
6	10,953,600	481	Success	Mesh small	1,285	27	217
7	5,897,510	508	Success	Mesh small	1,284	22	177
8	7,327,620	1,078	Success	Mesh small	1,283	30	241
9	367,957	1,428	Success	Mesh small	1,287	19	533
10	6,996,660	969	Success	Mesh small	1,290	21	169
11	6,163,850	841	Success	Mesh small	1,290	20	161
12	204,763	18,124	Success	Mesh small	1,247	37	1,185
13	650,128	1,280	Success	Mesh small	1,289	19	305
14	278,496	577	Success	Mesh small	1,289	11	89
15	1,545,050	12,700	Success	Mesh small	1,303	50	2,601
16	356,061	520	Success	Mesh small	1,288	10	81
17	244,938	799	Success	Mesh small	1,305	11	89
18	2,853,810	1,860	Success	Mesh small	1,312	26	209
19	614,144	–	Fail	24h Timeout	–	–	–
20	131,313	15,516	Success	Mesh small	1,346	22	1,673
21	3,115,360	17,052	Success	Mesh small	1,268	44	1,409
22	230,578	2,221	Success	Mesh small	1,240	13	105
23	152,889	746	Success	Mesh small	1,240	10	121
24	223,508	1,468	Success	Mesh small	1,244	23	277
25	222,550	937	Success	Mesh small	1,247	12	97
26	247,984	316	Success	Mesh small	1,341	9	73
27	391,768	–	Fail	24h Timeout	–	–	–
28	563,671	26,550	Success	Mesh small	1,353	15	301
29	11,012,600	–	Fail	24h Timeout	–	–	–
30	1,028,300	20,355	Success	Mesh small	1,350	34	2,041
31	1,084,200	68,432	Success	Mesh small	1,246	44	2,993
32	47,035,100	52,576	Success	Mesh small	1,350	38	1,825
33	30,894,600	–	Fail	24h Timeout	–	–	–
34	29,678,800	–	Fail	24h Timeout	–	–	–
35	28,579	36	Success	Mesh small	1,530	13	40

Table A3. Detailed results of TR_0 for the HA problem across all roads.

	TR_0						
Road number	Cost ($)	Time (s)	Status	Reason	Memory (mB)	True FC (#)	Surrogate FC (#)
1	262,740	82	Success	TR small	1,655	31	0
2	46,223	41	Success	TR small	1,472	20	0
3	560,863	343	Success	TR small	1,652	60	0
4	136,550	280	Success	TR small	1,652	39	0
5	8,876,460	243	Success	TR small	1,653	41	0
6	11,183,200	183	Success	TR small	1,652	31	0
7	6,035,530	248	Success	TR small	1,669	30	0
8	7,468,290	325	Success	TR small	1,697	30	0
9	377,635	1,001	Success	TR small	1,644	77	0
10	7,224,780	273	Success	TR small	1,651	18	0
11	6,160,100	330	Success	TR small	1,641	31	0
12	215,049	1,776	Success	TR small	1,721	90	0
13	719,287	753	Success	TR small	1,261	50	0
14	288,204	626	Success	TR small	1,277	30	0
15	1,630,980	2,419	Success	TR small	1,456	137	0
16	365,717	630	Success	TR small	1,282	31	0
17	250,031	776	Success	TR small	1,675	30	0
18	3,038,370	831	Success	TR small	1,652	31	0
19	848,507	2,741	Success	TR small	1,655	175	0
20	154,308	5,522	Success	TR small	1,407	198	0
21	3,214,460	6,772	Success	TR small	1,419	90	0
22	238,498	2,303	Success	TR small	1,659	31	0
23	153,501	481	Success	TR small	1,659	41	0
24	239,318	801	Success	TR small	1,658	40	0
25	223,173	163	Success	TR small	1,470	15	0
26	255,320	55	Success	TR small	1,440	31	0
27	386,715	70,888	Success	TR small	1,302	12	0
28	584,928	36,686	Success	TR small	1,330	60	0
29	11,077,600	81,564	Success	TR small	1,741	29	0
30	1,049,540	7,218	Success	TR small	1,280	161	0
31	1,126,310	14,163	Success	TR small	1,554	174	0
32	47,319,700	17,589	Success	TR small	1,455	121	0
33	30,860,100	–	Fail	24h T	–	–	–
34	30,781,500	65,649	Success	TR small	1,315	100	0
35	30,982	32	Success	TR small	1,718	21	0

Table A4. The detailed results of TR_ϵ for the HA problem across all roads.

	TR_ϵ						
Road number	Cost ($)	Time (s)	Status	Reason	Memory (mB)	True FC (#)	Surrogate FC (#)
1	266,295	63	Success	TR small	1,654	4	28
2	46,543	26	Success	TR small	1,469	4	18
3	562,267	256	Success	TR small	1,652	4	58
4	138,497	194	Success	TR small	1,653	4	38
5	8,957,230	182	Success	TR small	1,654	4	37
6	11,193,700	140	Success	TR small	1,651	4	28
7	6,052,440	186	Success	TR small	1,703	4	27
8	7,474,790	218	Success	TR small	1,689	4	27
9	378,078	679	Success	TR small	1,641	4	78
10	7,260,590	139	Success	TR small	1,651	2	17
11	6,194,380	259	Success	TR small	1,638	4	27
12	215,049	1,765	Success	TR small	1,723	4	88
13	722,599	833	Success	TR small	1,265	4	48
14	288,741	439	Success	TR small	1,281	4	28
15	1,652,720	1,623	Success	TR small	1,454	4	137
16	365,717	453	Success	TR small	1,284	4	28
17	250,031	584	Success	TR small	1,662	4	28
18	3,056,910	572	Success	TR small	1,653	4	28
19	848,700	2,775	Success	TR small	1,659	4	178
20	155,122	3,679	Success	TR small	1,417	4	198
21	3,214,870	4,256	Success	TR small	1,409	4	88
22	238,610	1,581	Success	TR small	1,659	4	28
23	153,748	396	Success	TR small	1,669	4	38
24	240,306	977	Success	TR small	1,661	4	38
25	223,173	175	Success	TR small	1,470	2	18
26	254,333	109	Success	TR small	1,440	4	27
27	395,426	54,644	Success	TR small	1,260	4	27
28	585,648	24,535	Success	TR small	1,331	4	58
29	11,077,600	80,765	Success	TR small	1,743	4	27
30	1,051,290	4,782	Success	TR small	1,273	4	158
31	1,128,230	8,189	Success	TR small	1,556	4	177
32	47,320,300	12,063	Success	TR small	1,458	4	128
33	30,860,100	–	Fail	24h T	–	–	–
34	30,547,400	68,147	Success	TR small	1,254	4	98
35	31,789	25	Success	TR small	1,721	4	18